PARALLEL BRANCH-AND-BOUND FOR EXACT PUMP SCHEDULING WITH CONSTANT-TIME HYDRAULIC BACKTRACKING

Jéssica Gomes Melo da Rocha[1], Albert Einstein Fernandes Muritiba[1], Ascânio Dias Araújo[1,2], Carlile Campos Lavor[3], and Michael Ferreira de Souza[1,3]

[1]Dept. of Statistics and Applied Mathematics, Federal University of Ceará, Campus do Pici, Fortaleza, CE 60455-760, Brazil. Corresponding author. Email: michael@ufc.br

[2]Dept. of Physics, Federal University of Ceará, Campus do Pici, Fortaleza, CE 60455-760, Brazil.

[3]IMECC, University of Campinas, Rua Sérgio Buarque de Holanda, 651, Campinas, SP 13083-859, Brazil.

ABSTRACT

Water utilities can reduce operating costs by choosing when pumps should run, but finding the best daily schedule is difficult because pump decisions, tank levels, and pressure limits are tightly coupled. This study examines whether exact optimization is now practical for realistic pump scheduling rather than relying only on heuristic search. We developed a parallel Branch-and-Bound framework for 24-hour pump scheduling in water distribution networks. The method splits the search tree across MPI processes, stores hydraulic snapshots so the solver can backtrack without repeating full simulations, and uses EPANET 3 to test feasibility for each candidate schedule. We evaluated the framework on the modified AnyTown benchmark with three pumps, three tanks, and several limits on the number of daily pump actuations. The solver reduced runtime by 29 to 115 times compared with a previous sequential exact method, depending on the operating constraint, and solved cases that previously required hours or days within minutes to a few hours on standard server hardware. In the least constrained case, it also found a lower-cost schedule than the best published heuristic result, improving energy cost by 3.7%. These findings show that exact day-ahead pump

scheduling is now practical for benchmark networks of realistic size when parallel computing and careful state management are used together. The proposed framework gives utilities and researchers a way to obtain provably optimal schedules, measure the quality of faster approximate methods, and study how operating constraints affect both cost and computational effort. The source code and benchmark data are publicly available.

PRACTICAL APPLICATIONS

Operating pumps is one of the largest daily energy costs in water utilities, so better schedules can lower expenses without changing physical infrastructure. In practice, many pump schedules are still based on operator experience or heuristic optimization tools that return a good answer but do not show whether it is the best feasible answer. This study shows that exact 24-hour pump scheduling can now be solved fast enough to support day-ahead decisions on a realistic benchmark network. Using a parallel Branch-and-Bound solver linked to EPANET 3, we cut solution times by 29 to 115 times compared with an earlier exact approach and improved the best published heuristic schedule by 3.7% in the least constrained case. For utilities and consultants, the main value is practical confidence: the method checks hydraulic limits directly, returns a provably optimal schedule for the modeled system, and reveals how much money may be left on the table by faster approximate methods. It can therefore support operational planning, benchmarking of commercial or in-house schedulers, and studies on how switching limits affect energy cost and system performance.

INTRODUCTION

Pump operations represent the dominant cost driver in water distribution networks (WDNs), with energy consumption for pumping comprising the largest share of operational expenses (Paola et al. 2025). As energy prices rise and environmental concerns intensify (Reis et al. 2023), developing efficient pump scheduling strategies has become both an economic and environmental imperative. Beyond cost minimization, effective scheduling influences system reliability, equipment longevity through controlled switching operations (Lansey and Awumah 1994; Costa et al. 2016), and service quality via consistent pressure maintenance and storage management (Reis et al. 2023).

The pump scheduling problem poses significant computational challenges. Decision variables are inherently discrete (pump on/off states), hydraulic constraints are nonlinear (pressure losses, pump curves), and operational requirements impose additional restrictions (tank level bounds, minimum pressures, maximum actuations). For a network with N_p pumps over a 24-hour horizon, the solution space encompasses 2^{24N_p} possible configurations. This combinatorial explosion, coupled with nonlinear hydraulic equations governing feasible solutions, places the problem firmly within the class of NP-hard mixed-integer nonlinear programs (MINLP) (Fooladivanda and Taylor 2018; Verleye and Aghezzaf 2016).

Early research employed classical mathematical programming methods such as Linear Programming, Dynamic Programming, and Nonlinear Programming (Brion and Mays 1991; Jowitt and Germanopoulos 1992). While these methods yielded foundational insights, they struggled with the high dimensionality and non-convexity inherent to realistic networks (Candelieri et al. 2018; Reis et al. 2023). Dynamic programming, though theoretically elegant, remained confined to small networks lacking switching constraints (Lansey and Awumah 1994).

These computational barriers precipitated a paradigm shift toward meta-heuristics. Genetic Algorithms (Savic et al. 1997; Mackle et al. 1995), Ant Colony Optimization (López-Ibáñez et al. 2008), and Particle Swarm Optimization (Al-Ani and Habibi 2012) emerged as dominant approaches (Reis et al. 2023), valued for their flexibility with discrete variables and complex constraint landscapes. However, all meta-heuristics share a fundamental limitation: they cannot guarantee optimality or quantify the gap to the global optimum (Costa et al. 2016; Reis et al. 2023). Recent innovations include Digital Harmony Search (DHS) (Paola et al. 2025), which incorporates real-time data with strict switching limits, and Smart Dynamic Local Search (Brás et al. 2026), employing duty-cycle formulations and perturbation mechanisms to escape local optima. Surrogate-based methods have also emerged to improve computational efficiency: Bayesian Optimization (Candelieri et al. 2018; Bagloee et al. 2018) for reduced simulation calls, artificial neural network metamodels (Broad et al. 2010; Rao and Alvarruiz 2007) for accelerating hydraulic evaluations during optimization, and physics-informed graph neural networks (Ashraf et al. 2024) for fast

emulation of hydraulic simulators. Nevertheless, meta-heuristics require extensive parameter tuning and exhibit sensitivity to initial conditions (Reis et al. 2023).

The computational tractability of exact methods for realistic pump scheduling was an open question until Costa et al. (2016) demonstrated that Branch-and-Bound (B&B) (Land and Doig 1960) could solve the problem for small-to-medium networks when appropriately constrained. Their key insight, which runs counter to typical assumptions, was that tighter operational constraints actually improve solver performance by enabling effective pruning of infeasible branches. In particular, limits on pump actuation frequency proved especially effective. However, their sequential implementation required hours or days of computation, limiting practical applicability. Recent efforts have also focused on developing tighter mathematical relaxations to improve B&B performance. For instance, Bonvin et al. (2021) proposed a tailored Mixed-Integer Linear Programming (MILP) relaxation using Outer Approximation (OA) to handle the non-convex hydraulic laws, providing an exact method applicable to both fixed and variable-speed pumps. While they focus on mathematical tightness to prune the search space, scalability remains a primary bottleneck for exact methods (Reis et al. 2023; Mala-Jetmarova et al. 2018).

The increasing accessibility of computing clusters and mature distributed-memory frameworks has renewed interest in exact approaches. Parallel B&B has proven successful in other combinatorial domains (Gendron and Crainic 1994; Li and Wah 1986; Hrga and Povh 2021), and decomposition methods like Generalized Benders Decomposition show promise for large networks with continuous relaxations (Verleye and Aghezzaf 2016). Yet parallel B&B for pump scheduling has remained limited (Reis et al. 2023), largely due to the computational overhead of repeated hydraulic simulations during tree traversal.

This work extends the foundational approach of Costa et al. (2016) with distributed-memory parallelism via MPI. We present a distributed-memory parallel B&B solver combining MPI-based task parallelism, hydraulic state persistence for $O(1)$ backtracking, and a two-level schedule representation that reduces branching factors. Integrated with EPANET 3's improved computational engine, our framework delivers four principal contributions:

1. A parallel task decomposition strategy with deterministic load balancing;
2. A snapshot-based state management system eliminating redundant hydraulic simulations during backtracking;
3. Cooperative pruning through non-blocking global bound synchronization;
4. Empirical validation demonstrates that exact 24-hour solutions become computationally tractable, with runtimes reduced from days to hours.

PROBLEM FORMULATION

The pump scheduling problem is formulated as a mixed-integer nonlinear program (MINLP). The goal is to determine optimal ON/OFF schedules for all pumps over a fixed planning horizon, minimizing total energy costs while satisfying hydraulic and operational constraints.

Objective Function

The objective minimizes total energy cost over the planning horizon:

$$\min_{\mathbf{x}} \quad z = \sum_{h=1}^{T} \sum_{j \in \mathcal{P}} C_{j,h} \cdot E_{j,h}(\mathbf{x}) \tag{1}$$

where $E_{j,h}(\mathbf{x})$ denotes the energy consumed by pump j during hour h, obtained from hydraulic simulation based on the full schedule $\mathbf{x}$.

Constraints

Nodal Pressure Limits. Pressures must remain within bounds to ensure adequate service without risking pipe damage:

$$P_i^{\min} \leq P_{i,h}(\mathbf{x}) \leq P_i^{\max} \quad \forall i \in \mathcal{I},\ \forall h \in \mathcal{H} \tag{2}$$

Tank Storage Bounds. Water levels must stay within operational limits to prevent overflow or depletion:

$$L_k^{\min} \leq L_{k,h}(\mathbf{x}) \leq L_k^{\max} \quad \forall k \in \mathcal{K},\ \forall h \in \mathcal{H} \tag{3}$$

Tank Recovery. To ensure day-to-day sustainability, final tank levels must meet or exceed initial levels:

$$L_{k,T}(\mathbf{x}) \geq L_{k,0} \quad \forall k \in \mathcal{K} \tag{4}$$

Actuation Limits. To reduce mechanical wear and maintenance costs, pump switching is restricted:

$$\sum_{h=2}^{T} |x_{j,h} - x_{j,h-1}| \leq N A_{\max} \quad \forall j \in \mathcal{P} \tag{5}$$

Hydraulic Model. The hydraulic state is governed by physical laws governing flow conservation, head losses, and pump characteristics (Bonvin et al. 2021). Let $q_{a,h} \in \mathbb{R}$ denote the flow rate (m^3/s) through arc a at time step h, and $H_{i,h} \geq 0$ denote the hydraulic head (m) at node i, defined as the sum of elevation and pressure head.

Flow conservation at each internal node requires:

$$\sum_{a \in \delta^-(j)} q_{a,h} - \sum_{a \in \delta^+(j)} q_{a,h} = D_{j,h}, \quad \forall j \in \mathcal{I} \setminus \mathcal{K},\ \forall h \in \mathcal{H} \tag{6}$$

where $\delta^-(j)$ and $\delta^+(j)$ denote arcs entering and leaving node j, respectively, and $D_{j,h} \geq 0$ is the water demand rate at node j during time step h.

For each pipe $\ell = (i, j) \in \mathcal{L}$, head loss follows the Hazen-Williams or Darcy-Weisbach relationship (Brown 2002; Burgschweiger et al. 2009a):

$$H_{i,h} - H_{j,h} = \Phi_\ell(q_{\ell,h}), \quad \forall \ell \in \mathcal{L},\ \forall h \in \mathcal{H} \tag{7}$$

where $\Phi_\ell(\cdot)$ is a nonlinear function with $\Phi_\ell(0) = 0$ and $\operatorname{sgn}(\Phi_\ell(q)) = \operatorname{sgn}(q)$ for all q.

For each pump $p = (i, j) \in \mathcal{P}$, the head increase across the pump depends on its status and flow (Burgschweiger et al. 2009b; Morsi et al. 2012):

$$H_{j,h} - H_{i,h} = \Psi_p(q_{p,h}) \cdot x_{p,h},$$

$$\forall p \in \mathcal{P},\ \forall h \in \mathcal{H} \quad (8)$$

where $\Psi_p(\cdot)$ is the pump curve derived from manufacturer data, and $x_{p,h} \in \{0, 1\}$ is the binary pump status.

Tank dynamics relate net inflow to water level change:

$$\sum_{a \in \delta^-(k)} q_{a,h} - \sum_{a \in \delta^+(k)} q_{a,h} = \frac{S_k}{\Delta t}(L_{k,h} - L_{k,h-1}),$$

$$\forall k \in \mathcal{K},\ \forall h \in \mathcal{H} \quad (9)$$

where S_k is the tank cross-sectional area (m^2) and $\Delta t = 3600$ s.

Rather than explicitly encoding the nonlinear constraints (6)–(9) in the optimization, our approach evaluates feasibility through hydraulic simulation. For each candidate schedule $\mathbf{x}$ visited during Branch-and-Bound traversal, EPANET 3 computes the hydraulic state (flows, heads, tank levels) by solving the full system (6)–(9) using the Global Gradient Algorithm (Todini and Pilati 1988). We then enforce constraints (2), (3), and (4) based on these simulated values. The actuation constraint (5) is enforced directly during schedule construction.

While this simulation-based approach ensures hydraulic accuracy, it introduces a computational bottleneck: each node in the Branch-and-Bound search tree requires a complete hydraulic simulation, and the number of nodes grows exponentially with the planning horizon. For a 24-hour schedule with $NA_{\max} = 3$, the search tree may contain billions of nodes, making sequential enumeration impractical. The following section addresses this challenge through parallelization and algorithmic optimizations that reduce both the number of nodes explored and the cost per node.

ENHANCED BRANCH-AND-BOUND

We present a distributed-memory parallel Branch-and-Bound (B&B) algorithm integrated with the EPANET 3 hydraulic engine. We adopt the two-level schedule representation proposed by Costa

et al. (2016), minimizing the branching factor, but enhance it with a Least Used Selection (LUS) strategy. To address scalability limitations of prior exact methods, we introduce three architectural innovations: cost-based pruning (which was not utilized in (Costa et al.)), MPI-based static load balancing with cooperative pruning, and a hydraulic snapshot mechanism for $O(1)$ state restoration.

Two-Level Schedule Representation

To mitigate the $2^{N_p \times T}$ combinatorial explosion, we adopt the variable separation strategy of Costa et al. (2016). A high-level decision variable $y_h \in \{0, 1, \ldots, N_p\}$ represents the number of active pumps at hour h. The low-level binary configuration $\mathbf{x}_h = \{x_{1,h}, \ldots, x_{N_p,h}\}$ is derived from y_h via a deterministic greedy mapping:

$$\mathcal{M} : (y_h, \mathbf{x}_{h-1}, \mathcal{R}) \rightarrow \mathbf{x}_h$$

where $\mathcal{R}$ denotes the remaining actuation budget for each pump. Fig. 1 illustrates this representation: the high-level schedule y_h specifies that 2, 3, 3, 2, 1, and 0 pumps should be active over six hours, which the mapping $\mathcal{M}$ converts into explicit binary pump states $x_{j,h}$.

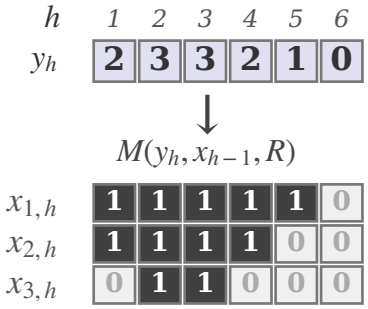

Fig. 1. Two-level schedule representation. The high-level variable y_h indicates how many pumps are active at each hour (here: 2, 3, 3, 2, 1, 0 over six hours for three pumps). The mapping $\mathcal{M}$ derives binary configurations $x_{j,h}$ respecting actuation constraints.

The mapping $\mathcal{M}$ selects pumps to satisfy the target count y_h while respecting actuation constraints $NA_{\max}$. Costa et al. (2016) employ a *sequential exhaustion* strategy: pump j is used exclusively until it exhausts its actuation budget, at which point the algorithm switches to pump

$j+1$. For instance, with one active pump, the sequence proceeds as $(1,0,0) \rightarrow (0,1,0) \rightarrow (0,0,1)$. Each pump operates until depleted before ceding to the next.

We propose *Least Used Selection* (LUS), which instead sorts pumps dynamically at each decision point by their remaining actuation budget. When switching pumps on, the algorithm prioritizes those with the most remaining on-switches; when switching off, it favors those with the most remaining off-switches. This greedy reordering serves a dual purpose: extending equipment life through balanced utilization across the pump station, while increasing the probability of finding feasible schedule extensions deeper in the search tree by preserving actuation flexibility.

Task Decomposition and Parallel Execution

We decompose the search tree into independent sub-problems to exploit distributed-memory parallelism. Each *task* is defined by a fixed prefix of the high-level schedule, $\mathbf{y}_{1:L} = \{y_1, \ldots, y_L\}$, where L denotes the decomposition level (typically $L = 8$). This yields $(N_p + 1)^L$ total tasks. Fig. 2 illustrates this decomposition: prefixes are enumerated at level L, and each resulting subtree is assigned to an MPI rank for independent exploration.

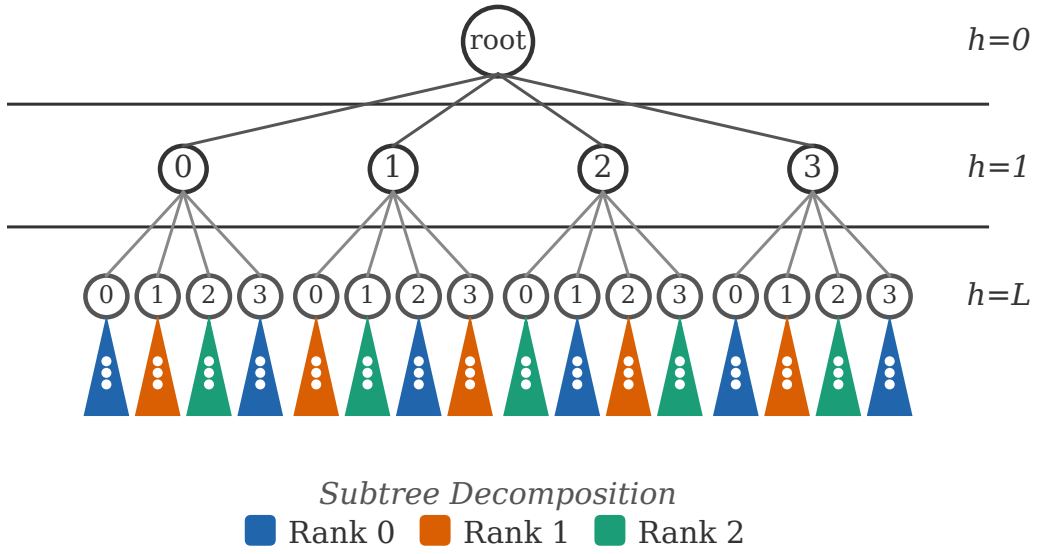

Fig. 2. Task decomposition by prefix. The search tree is partitioned at level L, yielding $(N_p + 1)^L$ independent subtrees distributed across MPI ranks. In this example, $L = 2$ and $N_p = 3$ generate 16 tasks assigned to 3 ranks via round-robin.

To achieve load balancing without the overhead of dynamic work stealing, we employ deterministic hashing: tasks are enumerated and assigned round-robin based on a hash of their prefix:

$$\text{Rank}(task_i) = \text{hash}(i) \mod N_{\text{procs}} \tag{10}$$

This approach mitigates the irregular structure of B&B trees, where some branches prune early while others require deep exploration.

Cooperative pruning leverages non-blocking global bound synchronization. Each process maintains a local upper bound UB_{local} and periodically exchanges information via MPI collective operations to compute the global minimum UB_{global}. This enables processes to prune sub-optimal branches based on solutions discovered by remote peers.

Hydraulic Snapshot Persistence

A critical bottleneck in simulation-based optimization is constraint evaluation cost. Standard Extended Period Simulation (EPS) requires simulating from $t = 0$ to $t = h$ to determine the hydraulic state at hour h. In depth-first traversal, backtracking from hour h to $h - 1$ traditionally necessitates complete re-simulation—an expensive operation repeated potentially billions of times during tree exploration.

EPANET 3 does not natively support state persistence for mid-simulation restoration. We therefore extended the hydraulic engine with a hierarchical snapshot mechanism that captures the minimal state required to resume simulation from any hour boundary. A *snapshot* S_h is a lightweight data structure storing:

Node states: hydraulic heads, demands, and outflows at all junctions;

Tank states: water volumes and surface areas;

Link states: flow rates, head losses, and valve/pump settings;

Pump energy accumulators: cumulative kWh consumption and operating costs;

Solver state: current simulation time, time pattern indices, and iterative solver variables.

Structural data (network topology, curves, control rules) remain in memory and are not duplicated. This design keeps snapshots compact while enabling complete state restoration.

During tree traversal, a snapshot is saved after each feasible hour:

$$S_h = \textsc{Save}(\textsc{Simulate}(S_{h-1}, \mathbf{x}_h)) \tag{11}$$

Backtracking to explore alternative branches at hour $h-1$ reduces to a memory copy, $S_{h-1} \rightarrow$ CurrentState, which executes in $O(1)$ relative to simulation time. The Save and Load operations in Algorithm 1 (lines 17 and 20) implement this mechanism. Table 1 shows that snapshot persistence delivers a 10.1× speedup by eliminating redundant hydraulic computations.

However, snapshot-based backtracking alone is insufficient for exact optimization if the hydraulic solver introduces path-dependent variability. EPANET's Global Gradient Algorithm uses an iterative Newton-Raphson method whose initial guess depends on the preceding hydraulic state. With the default convergence tolerance of $\epsilon = 10^{-3}$, identical pump configurations y_h reached via different search trajectories can produce energy discrepancies of up to 0.03 kWh. Such numerical noise violates the requirement that costs depend solely on the current configuration, undermining pruning guarantees.

To eliminate path-dependency, we tighten the solver tolerance to $\epsilon = 10^{-7}$ for all experiments. This ensures that the hourly cost $\sum_j C_{j,h} \cdot E_{j,h}(S_{h-1}, \mathbf{x}_h)$ is numerically consistent across all search paths reaching the same state, restoring strict cost determinism necessary for provable optimality.

Algorithm and Pruning Strategies

A key property enabling effective pruning is the *monotonicity* of the objective function. Because energy consumption $E_{j,h} \geq 0$ and electricity tariffs $C_{j,h} \geq 0$, each hourly cost contribution is non-negative. Consequently, the accumulated cost up to hour h, z_h,

$$z_h = \sum_{t=1}^{h} \sum_{j=1}^{N_p} C_{j,t} \cdot E_{j,t},$$

is monotonically non-decreasing as the search progresses: $z_h \leq z_{h+1} \leq \cdots \leq z_T$. This guarantees that if a partial solution's cost already meets or exceeds the best known complete solution, no extension of that partial solution can improve upon it. The branch can therefore be safely pruned without loss of optimality.

Our solver employs Depth-First Search (DFS), applying the following pruning criteria at each node (hour h) in order of computational cost:

1. **Actuation Viability:** Prune if $y_h \rightarrow \mathbf{x}_h$ mapping is infeasible within remaining budgets.
2. **Tank Bounds:** Prune if $L_{k,h} \notin [L_k^{\min}, L_k^{\max}]$ for any tank k.
3. **Cost Bound:** Prune if $z_h \geq UB_{\text{global}}$, since monotonicity ensures no feasible extension can yield a lower total cost.
4. **Pressure Limits:** Prune if $P_{i,h} < P_i^{\min}$ for any node i.
5. **Tank Recovery** (at $h = T$ only): Prune if $L_{k,T} < L_{k,0}$ for any tank k. Intermediate schedules are not pruned on this criterion because tank levels may still recover through subsequent pump operations.

This ordering prioritizes inexpensive checks (actuation, bounds) before hydraulic simulation, maximizing pruning efficiency. Algorithm 1 outlines the complete parallel procedure, which consists of two phases: a static distribution of search space prefixes to MPI ranks (Lines 1–8), followed by a recursive Depth-First Search (DFS) for each assigned task. Here, z^* denotes the global upper bound shared across processes (i.e., $z^* \equiv UB_{\text{global}}$), S_h represents the hydraulic snapshot at hour h, and c is the accumulated cost. The auxiliary function Map handles the greedy variable separation $\mathcal{M}$, while Sim performs hydraulic simulation to verify constraints: $\textsc{Sim}(\mathbf{y}_{1:L})$ simulates the entire prefix to initialize a task, and $\textsc{Sim}(h, \mathbf{x}_h)$ advances simulation by one hour. State management is handled by Save and Load, which enable $O(1)$ backtracking by storing and restoring hydraulic snapshots.

Our choice of DFS over the Breadth-First Search (BFS) strategy employed by Costa et al. (2016) is deliberate: DFS discovers complete feasible solutions more rapidly, establishing an upper

bound sooner for effective cost-based pruning. Furthermore, DFS requires only $O(\text{depth})$ memory compared to $O(\text{width})$ for BFS, which is essential for parallel scalability across distributed-memory systems.

Algorithm 1 Parallel Branch-and-Bound

Require: Horizon T, Decomposition level L, MPI rank r

$\mathcal{T} \leftarrow \{t \in \text{Prefixes}(L) \mid \text{hash}(t) \mod N_{\text{procs}} = r\}$
$z^* \leftarrow \infty$
for each prefix $\mathbf{y}_{1:L}$ in $\mathcal{T}$ **do**
    $S_L, c_L \leftarrow \textsc{Sim}(\mathbf{y}_{1:L})$
    if S_L feasible **then** DFS$(L+1, S_L, c_L)$
    end if
    Sync ▷ Non-blocking MPI bound exchange
end for
procedure DFS(h, S, c)
    if $c \geq z^*$ **then return**
    end if
    if $h > T$ **then**
        if tanks recovered **then** $z^* \leftarrow c$; Sync
        end ifreturn
    end if
    for $y = 0$ to N_p **do** ▷ y = target active pump count
        $\mathbf{x}_h \leftarrow \textsc{Map}(y)$
        if $\mathbf{x}_h$ invalid **then continue**
        end if
        $S_h, f \leftarrow \textsc{Sim}(h, \mathbf{x}_h)$
        if f **then**
            $\textsc{Save}(S_h)$
            DFS$(h{+}1, S_h, c{+}c_h)$
        end if
        $\textsc{Load}(S)$
    end for
end procedure

Having described our parallel B&B framework, we now validate its performance through systematic experiments on a standard benchmark network, examining scalability, ablation effects, and comparison with prior methods.

NUMERICAL EXPERIMENTS

The solver was implemented in C++17 with MPI for distributed computation. All experiments ran on a Linux server with an AMD EPYC 9554 64-Core Processor (128 logical cores with SMT enabled) and 384 GB RAM. Code was compiled using Intel oneAPI DPC++/C++ Compiler 2025.3.1 with `-O3 -march=native -flto` optimization flags. Hydraulic simulations used a customized

EPANET 3.0 library.

Case Study Network

To evaluate solver performance, we employ the AnyTown Modified network, a well-established benchmark in pump scheduling research. This network was originally proposed by Walski et al. (1987) and subsequently modified by Rao and Salomons (2007) to increase optimization complexity. The modified network comprises 44 pipes, 25 nodes, and three storage tanks located at nodes 65, 165, and 265. Water is supplied from a source at node 10 through three identical fixed-speed pumps operating in parallel (111, 222, and 333). All demand nodes follow the same temporal pattern, scaled by individual demand multipliers. Storage tanks operate within level bounds of 66.53 m (minimum) and 71.53 m (maximum). Fig. 3 shows the network topology used in our experiments, and Fig. 4 shows the hourly energy price pattern used for pump operating costs. This benchmark is particularly valuable for benchmarking because it captures the essential features of real-world systems (multiple pumps, storage facilities, and time-varying demands) while remaining tractable for comprehensive optimization studies.

Parameter Tuning

We tuned solver hyperparameters with Optuna (Akiba et al. 2019) by sweeping the number of MPI processes (N_{procs}), decomposition level (L), and synchronization interval (S, the number of simulation steps between global bound exchanges) on a 16-hour instance ($NA_{\max} = 2$). The best configuration was $N_{\text{procs}} = 128$, $L = 8$, and $S = 32768$ simulation steps, and we use it for all subsequent experiments.

Ablation Study

We evaluated each algorithmic component by selectively disabling features on a reduced instance ($T = 24$ hours, $NA_{\max} = 1$) using 128 MPI processes. Table 1 summarizes the results.

The snapshot mechanism is the largest contributor in our ablation, delivering a 10.1× speedup by eliminating redundant hydraulic computations during backtracking. Cost-based pruning contributes a 5.8× improvement by cutting sub-optimal branches early. Task shuffling provides a 3.1× speedup

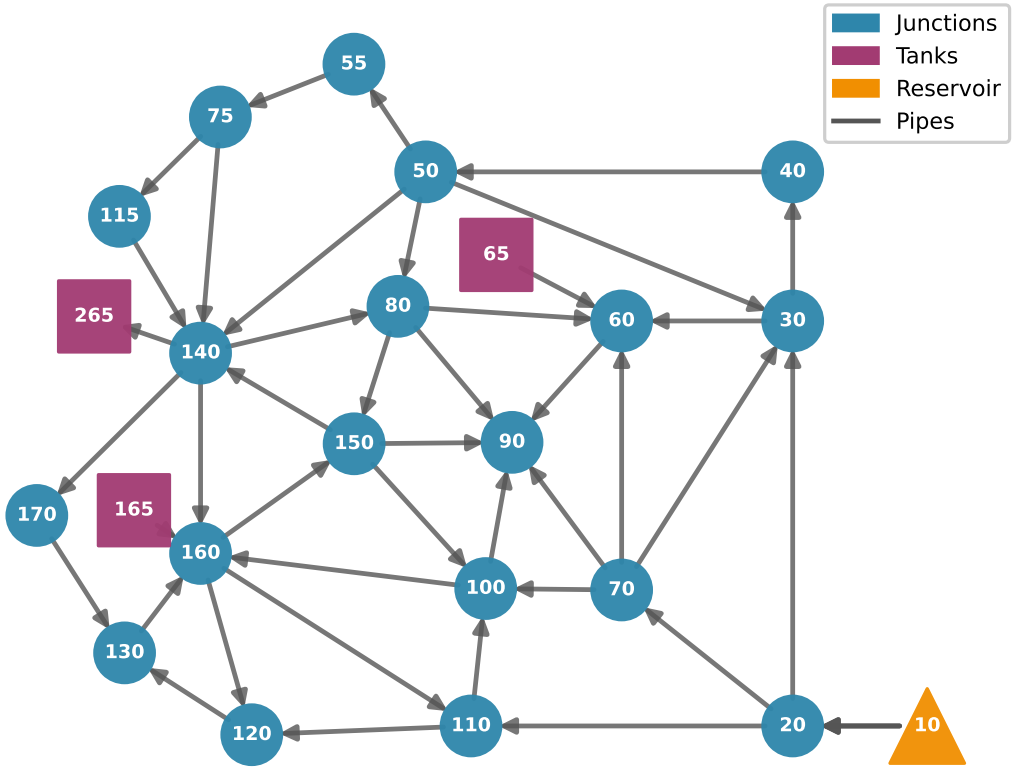

Fig. 3. Topology of the AnyTown Modified network. The network includes a reservoir (node 10), three storage tanks (nodes 65, 165, 265), and 19 demand nodes connected by 44 pipes. Three parallel pumps supply water from the reservoir.

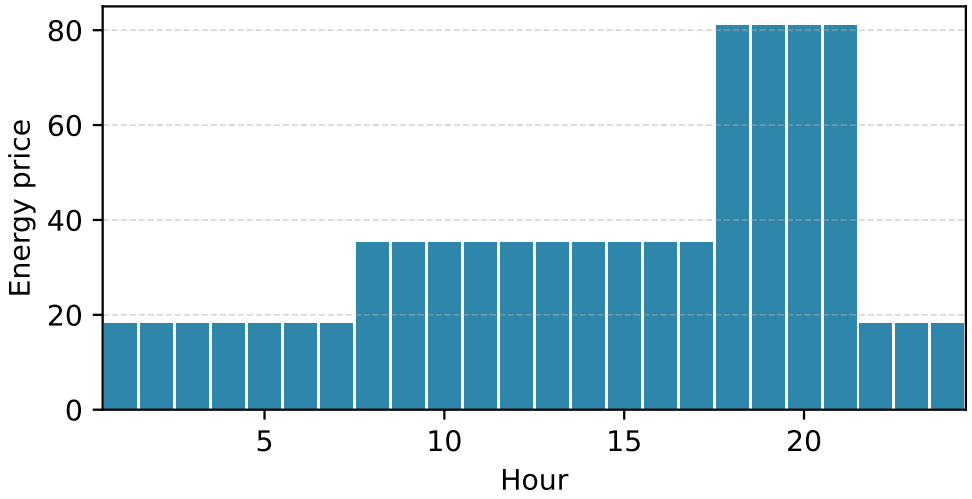

Fig. 4. Hourly energy price pattern used for pump operating cost calculations in the AnyTown Modified network.

TABLE 1. Ablation study results ($T = 24$, $NA_{\max} = 1$, $N_{\text{procs}} = 128$). Slowdown shows how much slower each configuration runs compared to "All Features" (higher = slower).

Configuration	Time (s)	Slowdown	Cost ($)
All Features	5.47	1.0×	3567.99
No Snapshots	55.47	10.1×	3567.99
No Cost Pruning	31.86	5.8×	3567.99
No Pump Sorting	3.11	0.6×	3582.98
No Task Shuffle	16.95	3.1×	3567.99

by improving load balance across processes.

Notably, disabling pump sorting yields the *fastest* runtime (3.11s) because strict pruning quickly eliminates branches violating actuation constraints. This reverts to the sequential exhaustion strategy of Costa et al. However, this configuration fails to find the true optimum ($3582.98 vs $3567.99). LUS finds better solutions by dynamically reordering pump selection based on remaining actuation budgets, which explores regions of the search space that sequential exhaustion prunes prematurely. By preserving flexibility across all pumps, LUS maintains access to combinations that may yield lower costs later in the horizon, albeit at the expense of exploring more branches.

Parallel Scalability

TABLE 2. Parallel scaling metrics ($T = 24$, $NA_{\max} = 1$).

N_{procs}	Time (s)	Speedup	Eff. (%)
1	85.08	1.00	100.0
2	46.13	1.84	92.2
4	23.19	3.67	91.7
8	12.35	6.89	86.1
16	7.47	11.39	71.2
32	4.26	20.00	62.5
64	3.18	26.76	41.8
128	3.59	23.69	18.5

Fig. 5 and Table 2 present scaling results for a 24-hour instance ($NA_{\max} = 1$), varying MPI processes from 1 to 128. The solver achieves 26.76× speedup with 64 processes, reducing wall-clock time from 85.08s (sequential) to 3.18s. Near-linear scaling is observed up to 8 processes (86.1% efficiency), after which efficiency drops due to load imbalance inherent to B&B tree pruning.

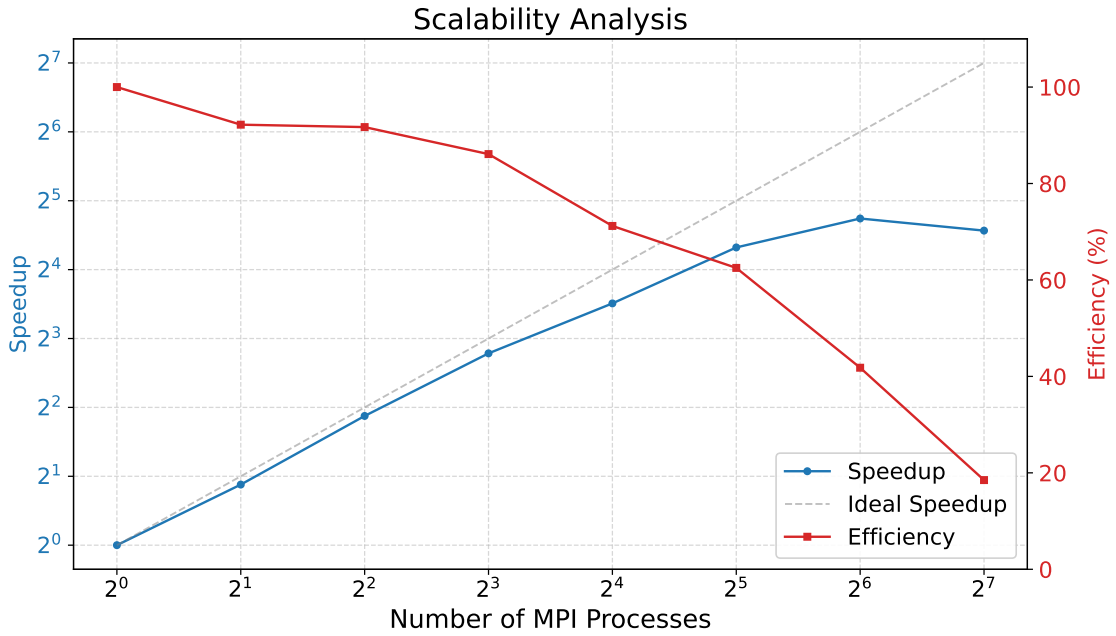

Fig. 5. Parallel scalability on AnyTown Modified ($T = 24$, $NA_{\max} = 1$).

The load imbalance factor, defined as $(t_{\max} - t_{\text{avg}})/t_{\text{avg}}$, grows from 8.3% at 4 processes to 102.6% at 64 processes. At 128 processes, performance decreases by 13% (wall time 3.59s vs 3.18s at 64 processes) due to severe load imbalance (165.2%), where coordination overhead exceeds the benefit of additional parallelism for this problem size. Despite this, we retain $N_{\text{procs}} = 128$ for subsequent experiments because larger problem instances ($NA_{\max} = 2, 3$) exhibit substantially better load balance (see Table 3), where the additional processes provide meaningful speedup.

24-Hour Optimization

We solved the complete 24-hour problem under varying actuation constraints with $N_{\text{procs}} = 128$ ($L = 8$, $S = 32768$). Table 3 presents the results, and Fig. 6 shows the corresponding tank level and nodal pressure trajectories over the 24-hour horizon. The figure displays tank water levels (top panel) for tanks 65, 165, and 265 with operational bounds $L_{\min} = 66.53$ m and $L_{\max} = 71.53$ m, and nodal pressures (bottom panel) at representative nodes 55, 90, and 170 with their respective minimum thresholds (42, 51, and 30 m). Shaded regions indicate infeasible operating ranges.

The results reveal a clear trade-off between operational flexibility and computational cost. Restricting actuations to $NA_{\max} = 1$ reduces the search space substantially, enabling solution in

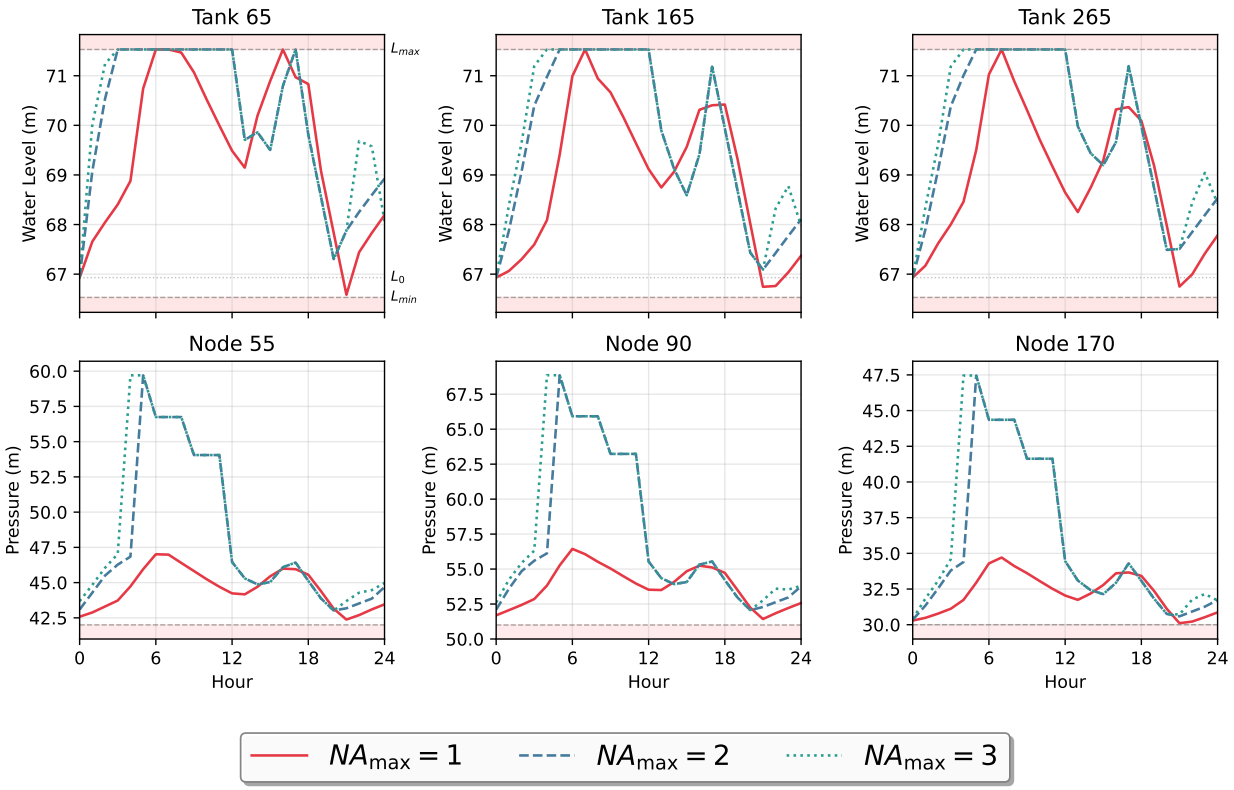

Fig. 6. Hydraulic trajectories for optimal schedules under different actuation limits ($NA_{\max} = 1, 2, 3$). Top: tank water levels; dashed lines show bounds, dotted line shows initial level. Bottom: nodal pressures; dashed lines show minimum thresholds.

TABLE 3. Results for 24-hour optimization ($T = 24$) on AnyTown Modified. Load imbalance is defined as $(t_{\max} - t_{\text{avg}})/t_{\text{avg}}$ across MPI ranks.

$NA_{\max}$	Cost ($)	Time (s)	Load Imb. (%)
1	3,567.99	3.69	172.0
2	3,472.89	493.11	23.6
3	3,443.46	10011.23	12.9

under 4 seconds, but at a 3.6% higher cost ($3,567.99) relative to the 3-actuation case. However, this configuration exhibits severe load imbalance (172.0%), indicating that most of the search space is pruned early by actuation constraints, leaving few tasks with deep exploration. Allowing $NA_{\max} = 3$ yields the lowest cost of $3,443.46, but requires 10,011s (≈2.8h). Interestingly, this configuration achieves the best load balance (12.9% imbalance) because the larger search space distributes work more evenly across processes. The $NA_{\max} = 2$ case achieves 99.1% of the optimal

cost in 4.9% of the time required for $NA_{\max} = 3$, reaching \$3,472.89 in 493s (≈8.2min) with moderate imbalance (23.6%).

Pruning Effectiveness

Table 4 details the contribution of each pruning criterion to the search space reduction. The multi-criteria pruning strategy proves highly effective, eliminating 60–72% of explored nodes before reaching terminal states.

TABLE 4. Pruning statistics (% of nodes) by constraint.

	$NA_{\max}$		
Criterion	1	2	3
Actuation	61.9	43.9	23.7
Cost bound	9.1	19.9	30.0
Tank levels†	0.0	0.0	0.0
Pressure	0.8	3.8	9.0
Total pruned	71.8	67.6	62.7
Feasible	28.2	32.4	37.3
Nodes (total)	7.9M	1.4B	22.4B

†EPANET enforces tank bounds via head clamping; see text.

Actuation constraints dominate pruning for $NA_{\max} = 1$ and $NA_{\max} = 2$, accounting for up to 62% of all explored nodes. This validates the insight of (Costa et al.) that tighter operational constraints improve solver performance. As actuation limits relax ($NA_{\max} = 3$), cost-based pruning becomes increasingly important (30.0%), reflecting more opportunities for the global bound to eliminate sub-optimal branches. Pressure violations also become more prominent as the search depth increases, reaching 9.0% for $NA_{\max} = 3$.

The lack of "tank levels" pruning is expected for this case study. In EPANET, tank levels are enforced as hydraulic boundary conditions: when a tank reaches its maximum level, its head is clamped at $L_k^{\max}$ and any additional net inflow is blocked by temporarily closing incident links, so the simulated level remains exactly at the limit rather than exceeding it. As a result, schedules that "appear" to keep tanks full while pumps operate do not violate the bound; the hydraulic solution simply redistributes flow and/or reduces pump flow under the higher head. Because this behavior is intrinsically handled by the simulator, an explicit pre-simulation pruning rule based on anticipated tank overfill would be both unreliable (risking false pruning) and redundant (the constraint cannot be

violated in the hydraulic state). Consequently, pruning is driven primarily by actuation feasibility, cost bounds, and pressure violations.

Comparative Analysis

We compare our framework with the sequential B&B of Costa et al. (2016), the Genetic Algorithm of Cimorelli et al. (2020), and the Digital Harmony Search (DHS) of Paola et al. (2025) on AnyTown Modified. All reference solutions were re-evaluated using EPANET 3.0 to ensure consistency; each remained hydraulically feasible with costs matching original reports within 0.1% (differences attributable to minor variations between EPANET 2 and EPANET 3 hydraulic engines).

Figs. 7–9 present detailed pump schedules for all methods at each actuation limit. These visualizations show that our schedules differ from prior approaches in pump timing and activation sequences while achieving superior cost performance.

Our EPANET-BB framework achieves the lowest cost across all actuation limits tested, improving upon both exact and meta-heuristic methods. For $NA_{\max} = 1$, EPANET-BB finds \$3,567.99, reducing costs by 1.8% compared to Cimorelli et al.'s GA (\$3,634.67). For $NA_{\max} = 2$, our solution (\$3,472.89) improves by 3.0% over Cimorelli et al. (\$3,580.11). For $NA_{\max} = 3$, EPANET-BB achieves \$3,443.46, a 3.7% improvement over Cimorelli et al.'s best-reported result (\$3,575.54). These improvements demonstrate the value of exhaustive search with effective pruning.

Computational performance shows substantial improvement over sequential B&B: speedups of 29 to 115 times were observed compared to the sequential implementation of (Costa et al.). The most constrained case ($NA_{\max} = 1$) reduced computation time from 425 seconds to 3.69 seconds (115× speedup). The moderately constrained case ($NA_{\max} = 2$) decreased from 10.3 hours to 8.2 minutes (75× speedup). The least constrained case ($NA_{\max} = 3$) reduced computation from 81 hours to 2.8 hours (29× speedup). These performance enhancements demonstrate that exact optimization can be applied within sub-hour to few-hour timeframes suitable for day-ahead operational planning, offering theoretical guarantees that are not inherent to meta-heuristic approaches.

Discussion

Our results reveal several noteworthy findings and limitations that merit discussion.

Maximum Actuations: 1

Source	Cost ($)	Time (s)	Pump	SW	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Costa et al. (2016)	**3916.98**	425.00	P1	3	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●			●	●	●
			P2	2		●	●																					
			P3	2												●	●	●										
Cimorelli et al. (2020)	**3634.67**	663.00	P1	4	●	●	●	●	●	●	●	●	●	●	●	●				●	●							
			P2	3	●																					●	●	●
			P3	4	●										●	●	●	●	●	●	●	●						
De Paola et al. (2025)	**3911.52**	72.00	P1	2				●																				
			P2	3	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●			●	●	●
			P3	2												●	●	●										
EPANET-BB	**3567.99**	3.69	P1	3	●	●	●	●	●	●	●															●	●	●
			P2	2					●	●	●	●	●	●	●	●	●	●	●	●								
			P3	2														●	●	●	●	●						

Fig. 7. Pump schedule comparison for $NA_{\max} = 1$. EPANET-BB achieves the lowest cost ($3,567.99), improving upon Cimorelli et al.'s GA ($3,634.67) by 1.8%.

Maximum Actuations: 2

Source	Cost ($)	Time (s)	Pump	SW	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Costa et al. (2016)	**3618.59**	36914.00	P1	5	●	●	●	●	●	●	●	●	●		●	●	●	●	●	●	●	●				●	●	●
			P2	4		●	●								●	●	●											
			P3	4															●		●							
Cimorelli et al. (2020)	**3580.11**	663.00	P1	6	●		●	●	●	●	●	●								●	●							
			P2	4											●	●	●	●								●	●	
			P3	4	●	●	●								●	●	●	●	●	●	●	●						
De Paola et al. (2025)	**3606.22**	904.00	P1	5	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	●
			P2	4		●	●								●	●	●	●	●									
			P3	2																	●							
EPANET-BB	**3472.89**	493.11	P1	5	●	●	●											●	●	●	●				●	●	●	●
			P2	4	●	●	●	●	●	●	●	●	●	●	●	●				●	●							
			P3	4							●	●	●	●	●	●					●							

Fig. 8. Pump schedule comparison for $NA_{\max} = 2$. EPANET-BB achieves the lowest cost ($3,472.89), improving upon Cimorelli et al.'s GA ($3,580.11) by 3.0%.

Maximum Actuations: 3

Source	Cost ($)	Time (s)	Pump	SW	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Costa et al. (2016)	**3578.67**	292032.00	P1	6	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	
			P2	6		●		●							●	●	●	●	●									
			P3	4																	●					●		
Cimorelli et al. (2020)	**3575.54**	663.00	P1	8	●	●	●	●	●	●	●	●			●	●	●		●	●	●					●	●	
			P2	8	●			●							●	●	●		●	●	●							
			P3	6												●	●					●				●		
De Paola et al. (2025)	**3577.40**	960.00	P1	6	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	
			P2	6	●		●								●	●	●	●	●									
			P3	4																	●					●		
EPANET-BB	**3443.46**	10011.23	P1	6	●						●	●	●	●	●	●					●							
			P2	6	●	●	●											●	●	●	●				●	●		
			P3	6	●	●	●	●	●	●	●	●	●	●	●	●				●	●					●	●	

Fig. 9. Pump schedule comparison for $NA_{\max} = 3$. EPANET-BB achieves the lowest cost ($3,443.46), improving upon Cimorelli et al.'s GA ($3,575.54) by 3.7%.

LUS Trade-off. The ablation study uncovered an unexpected trade-off: disabling pump sorting (LUS) produces the fastest runtime but yields suboptimal solutions. This occurs because sequential exhaustion aggressively prunes the search space by depleting each pump's actuation budget before considering alternatives, which happens to align well with the pruning criteria but misses better configurations that require distributing actuations across pumps. LUS sacrifices some pruning efficiency to maintain flexibility, ultimately finding the global optimum at modest computational cost.

Scalability Considerations. The AnyTown Modified network has three pumps, yielding a search space of $4^{24} \approx 2.8 \times 10^{14}$ high-level configurations for the full 24-hour problem. Networks with more pumps will face exponentially larger search spaces. Our framework's effectiveness depends critically on the actuation constraint $NA_{\max}$, which provides the dominant pruning mechanism. For networks where operational constraints are looser, additional strategies such as bound tightening, symmetry breaking, or hierarchical decomposition may be necessary.

Parallel Efficiency. Load imbalance remains the primary barrier to parallel efficiency, degrading from 86% at 8 processes to 42% at 64 processes for the $NA_{\max} = 1$ case. Interestingly, larger problem instances ($NA_{\max} = 3$) achieve better load balance (12.9% imbalance) because the search

space is more uniformly distributed. Dynamic work stealing could improve efficiency for highly constrained instances where static decomposition produces uneven subtrees.

Generalizability. While our experiments focus on a single benchmark network, the algorithmic innovations—snapshot persistence, LUS, and cooperative pruning—are network-agnostic and should transfer to other fixed-speed pump scheduling problems. Variable-speed pumps introduce continuous decision variables that would require different solution approaches.

CONCLUSION

We presented a high-performance Branch-and-Bound framework for pump scheduling in water distribution networks. Our three key innovations are MPI-based parallelism, hydraulic snapshot persistence, and balanced pump selection via LUS. These bridge the gap between theoretical optimality and practical applicability.

Our experiments demonstrate that exact optimization of 24-hour pump schedules is now feasible in operational timeframes. The solver achieves speedups of 29–115× over sequential implementations, reducing computation from hours to minutes while maintaining optimality guarantees. Notably, our results improve upon the best-known solutions from meta-heuristics, achieving \$3,443.46 for $NA_{\max} = 3$—a 3.7% improvement over Cimorelli et al.'s GA result (\$3,575.54). The ablation study confirms that snapshot persistence delivers the most significant performance gain (10.1×), validating our focus on eliminating redundant hydraulic simulations.

Two directions merit further investigation. First, dynamic work stealing could improve parallel efficiency beyond the 42% observed at 64 processes by addressing load imbalance inherent to B&B tree exploration. Second, extending the framework to larger networks with more pumps will require additional pruning strategies or hierarchical decomposition to manage the exponential growth in search space.

DATA AVAILABILITY STATEMENT

Some or all data, models, or code generated or used during the study are available in a repository online in accordance with funder data retention policies. The EPANET-BB source code, benchmark

network files, experiment configurations, and processed result tables used to support the findings of this study are available at https://github.com/michaelsouza/epanet-bb.

ACKNOWLEDGMENTS

This research was partially funded by the Brazilian research agencies FAPESP (Grant Nos. 2013/07375-0, 2023/08706-1, and 2024/00923-6) and CNPq (Grant Nos. 305227/2022-0, 404616/2024-0, and 402609/2025-5). Michael Ferreira de Souza, Albert Einstein Fernandes Muritiba, and Ascânio Dias Araújo acknowledge financial support from CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior), CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), and FUNCAP (Fundação Cearense de Apoio ao Desenvolvimento Científico e Tecnológico). Jéssica Gomes Melo da Rocha acknowledges a Master's scholarship from FUNCAP. The funding agencies had no role in the study design; data collection, analysis, or interpretation; manuscript preparation; or the decision to submit the article for publication.

NOTATION

The following symbols are used in this paper:

$\mathcal{A}$ = set of network arcs;

$C_{j,h}$ = electricity tariff for pump j at time step h (\$/kWh);

$D_{j,h}$ = water demand at node j and time step h (m^3/s);

$E_{j,h}$ = energy consumed by pump j during time step h (kWh);

$\mathcal{H}$ = set of time steps in the planning horizon;

$H_{i,h}$ = hydraulic head at node i and time step h (m);

$\mathcal{I}$ = set of network nodes;

$\mathcal{K}$ = set of storage tanks;

L = decomposition level used for task partitioning;

$L_{k,h}$ = water level in tank k at time step h (m);

$\mathcal{L}$ = set of pipes;

$NA_{\max}$ = maximum number of pump state transitions over the planning horizon;

N_p = number of pumps;

$\mathcal{P}$ = set of pumps;

$P_{i,h}$ = pressure at node i and time step h (m);

S_k = tank cross-sectional area (m^2);

T = number of time steps in the planning horizon;

UB_{global} = global upper bound on total operating cost;

UB_{local} = local upper bound on total operating cost;

$q_{a,h}$ = flow rate through arc a at time step h (m^3/s);

$x_{j,h}$ = binary status of pump j at time step h;

y_h = number of active pumps at time step h;

Δt = simulation time-step duration (s);

$\Phi_\ell(\cdot)$ = head-loss relation for pipe ℓ;

$\Psi_p(\cdot)$ = head-gain relation for pump p.

REFERENCES

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). "Optuna: A next-generation hyperparameter optimization framework." *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '19, ACM, 2623–2631 (July).

Al-Ani, D. and Habibi, S. (2012). "Optimal pump operation for water distribution systems using a new multi-agent particle swarm optimization technique with EPANET." *2012 25th IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, IEEE, 1–6 (April).

Ashraf, I., Strotherm, J., Hermes, L., and Hammer, B. (2024). "Physics-informed graph neural networks for water distribution systems." *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(20), 21905–21913.

Bagloee, S. A., Asadi, M., and Patriksson, M. (2018). "Minimization of water pumps' electricity usage: A hybrid approach of regression models with optimization." *Expert Systems with Applications*, 107, 222–242.

Bonvin, G., Demassey, S., and Lodi, A. (2021). "Pump scheduling in drinking water distribution networks with an LP/NLP-based branch and bound." *Optimization and Engineering*, 22(3), 1275–1313.

Brás, M., Moura, A., and Andrade-Campos, A. (2026). "A novel hybrid optimization approach for cost-efficient pump scheduling in water supply systems." *Omega*, 138, 103378.

Brion, L. M. and Mays, L. W. (1991). "Methodology for optimal operation of pumping stations in water distribution systems." *Journal of Hydraulic Engineering*, 117(11), 1551–1569.

Broad, D. R., Maier, H. R., and Dandy, G. C. (2010). "Optimal operation of complex water distribution systems using metamodels.." *Journal of Water Resources Planning and Management*, 136(4), 433–443.

Brown, G. O. (2002). "The history of the Darcy-Weisbach equation for pipe flow resistance." *Environmental and Water Resources History*, Reston, VA, American Society of Civil Engineers, 34–43 (October).

Burgschweiger, J., Gnädig, B., and Steinbach, M. C. (2009a). "Nonlinear programming techniques

for operative planning in large drinking water networks." *Open Applied Mathematics Journal*, 3(1), 14–28.

Burgschweiger, J., Gnädig, B., and Steinbach, M. C. (2009b). "Optimization models for operative planning in drinking water networks." *Optimization and Engineering*, 10(1), 43–73.

Candelieri, A., Perego, R., and Archetti, F. (2018). "Bayesian optimization of pump operations in water distribution systems." *Journal of Global Optimization*, 71(1), 213–235.

Cimorelli, L., D'Aniello, A., and Cozzolino, L. (2020). "Boosting genetic algorithm performance in pump scheduling problems with a novel decision-variable representation." *Journal of Water Resources Planning and Management*, 146(5), 04020023.

Costa, L. H. M., de Athayde Prata, B., Ramos, H. M., and de Castro, M. A. H. (2016). "A branch-and-bound algorithm for optimal pump scheduling in water distribution networks." *Water Resources Management*, 30(3), 1037–1052.

Fooladivanda, D. and Taylor, J. A. (2018). "Energy-optimal pump scheduling and water flow." *IEEE Transactions on Control of Network Systems*, 5(3), 1016–1026.

Gendron, B. and Crainic, T. G. (1994). "Parallel branch-and-bound algorithms: Survey and synthesis." *Operations Research*, 42(6), 1042–1066.

Hrga, T. and Povh, J. (2021). "MADAM: a parallel exact solver for max-cut based on semidefinite programming and ADMM." *Computational Optimization and Applications*, 80(2), 347–375.

Jowitt, P. W. and Germanopoulos, G. (1992). "Optimal pump scheduling in water-supply networks." *Journal of Water Resources Planning and Management*, 118(4), 406–422.

Land, A. H. and Doig, A. G. (1960). "An automatic method of solving discrete programming problems." *Econometrica*, 28(3), 497–520.

Lansey, K. E. and Awumah, K. (1994). "Optimal pump operations considering pump switches." *Journal of Water Resources Planning and Management*, 120(1), 17–35.

Li, G.-J. and Wah, B. W. (1986). "Coping with anomalies in parallel branch-and-bound algorithms." *IEEE Transactions on Computers*, C-35(6), 568–573.

López-Ibáñez, M., Prasad, T. D., and Paechter, B. (2008). "Ant colony optimization for optimal

control of pumps in water distribution networks." *Journal of Water Resources Planning and Management*, 134(4), 337–346.

Mackle, G., Savic, D. A., and Walters, G. A. (1995). "Application of genetic algorithms to pump scheduling for water supply." *First International Conference on Genetic Algorithms in Engineering Systems: Innovations and Applications (GALESIA)*, Vol. 1995, IEE, 400–405.

Mala-Jetmarova, H., Sultanova, N., and Savic, D. (2018). "Lost in optimisation of water distribution systems? a literature review of system design." *Water*, 10(3), 307.

Morsi, A., Geißler, B., and Martin, A. (2012). "Mixed integer optimization of water supply networks." *Mathematical Optimization of Water Networks*, Springer, 35–54.

Paola, F. D., Pugliese, F., Fontana, N., and Giugni, M. (2025). "A new digital harmony search algorithm for optimizing pump scheduling in water distribution networks." *Water Research X*, 27, 100300.

Rao, Z. and Alvarruiz, F. (2007). "Use of an artificial neural network to capture the domain knowledge of a conventional hydraulic simulation model." *Journal of Hydroinformatics*, 9(1), 15–24.

Rao, Z. and Salomons, E. (2007). "Development of a real-time, near-optimal control process for water-distribution networks." *Journal of Hydroinformatics*, 9(1), 25–37.

Reis, A. L., Lopes, M. A. R., Andrade-Campos, A., and Antunes, C. H. (2023). "A review of operational control strategies in water supply systems for energy and cost efficiency." *Renewable and Sustainable Energy Reviews*, 175, 113140.

Savic, D. A., Walters, G. A., and Schwab, M. (1997). "Multiobjective genetic algorithms for pump scheduling in water supply." *Workshop on Evolutionary Computing*, Springer, 227–235.

Todini, E. and Pilati, S. (1988). "A gradient algorithm for the analysis of pipe networks." *Computer Applications in Water Supply: Vol. 1—Systems Analysis and Simulation*, B. Coulbeck and C.-H. Orr, eds., Research Studies Press Ltd., Taunton, UK, 1–20.

Verleye, D. and Aghezzaf, E.-H. (2016). "Generalized benders decomposition to reoptimize water production and distribution operations in a real water supply network." *Journal of Water*

Resources Planning and Management, 142(2), 04015059.

Walski, T. M., Brill, E. D., Gessler, J., Goulter, I. C., Jeppson, R. M., Lamey, K., Lee, H. L., Liebman, J. C., Mayo, L., Morgan, D. R., and Ormsbee, L. (1987). "Battle of the network models: epilogue." *Journal of Water Resources Planning and Management*, 113(2), 191–203.